Module 9 — Working with Pandas

Why We Are Learning Pandas

In previous lectures we learned **NumPy**, which is very powerful for numerical computation.

However, real-world datasets are rarely pure numerical matrices.

Real datasets contain:

- Names
- Dates
- Categories
- Missing values
- Mixed data types

Example dataset:

Name	Age	Salary
Rahul	25	50000
Priya	28	65000
Aman	24	48000

NumPy struggles to handle this type of **tabular data efficiently**.

This is where **Pandas** becomes extremely useful.

What is Pandas?

Pandas is a **Python library used for data analysis and data manipulation**.

It provides powerful tools to:

- Organize structured data
- Clean messy datasets
- Perform statistical analysis
- Prepare data for machine learning

Pandas works **on top of NumPy**.

This means:

NumPy → numerical engine
Pandas → data analysis toolkit

Why Pandas is Important

Pandas is widely used in:

Data Science

- Cleaning and preparing datasets

Data Analytics

- Analyzing business data

Finance

- Stock market analysis

Machine Learning

- Preparing datasets for models

Research

- Handling large experimental datasets

Key Features of Pandas

• Handles tabular data easily
• Supports missing values
• Powerful indexing and filtering
• Built-in statistical functions
• Easy file reading (CSV, Excel, SQL)

Two Core Data Structures in Pandas

Pandas mainly provides two data structures:

1. Series
2. DataFrame

1. Pandas Series

A **Series** is a one-dimensional labeled array.

It is similar to:

- A column in a table
- A NumPy array with labels

Structure of a Series:

Index → labels
Values → actual data

Example:

Index	Value
0	10
1	20
2	30

```
import pandas as pd

# Creating a Series

data = pd.Series([10, 20, 30, 40])

print(data)
```

```
0    10
1    20
2    30
3    40
dtype: int64
```

Custom Index in Series

We can also define our own labels for the Series.

```
data = pd.Series([100, 200, 300], index=["A", "B", "C"])

print(data)
```

```
A    100
B    200
C    300
dtype: int64
```

Accessing Series Values

We can access elements using the index.

```
print(data["A"])
```

```
100
```

2. Pandas DataFrame

A **DataFrame** is a two-dimensional table.

It is similar to:

• Excel spreadsheet
• SQL table
• Dataset used in data science

Example structure:

Name	Age	Salary
Rahul	25	50000
Priya	28	65000
Aman	24	48000

```
# Creating a DataFrame manually

data = {
    "Name": ["Rahul", "Priya", "Aman"],
    "Age": [25, 28, 24],
    "Salary": [50000, 65000, 48000]
}

df = pd.DataFrame(data)

print(df)
```

```
    Name  Age  Salary
0  Rahul   25   50000
1  Priya   28   65000
2   Aman   24   48000
```

Observations

In a DataFrame:

Rows → represent observations
Columns → represent variables

Each column can have a different data type:

Name → string
Age → integer
Salary → integer

SESSION 2 — Reading and Writing Data

In real projects, we rarely create datasets manually.

Instead we load datasets from files such as:

• CSV files
• Excel files
• Databases

Pandas provides functions to easily read and write these files.

Reading CSV Files

CSV stands for:

Comma Separated Values

Syntax:

pd.read_csv("filename.csv")

```
# Example dataset

df = pd.read_csv("sample_data/california_housing_test.csv")

print(df)
```

```
      longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0       -122.05     37.37                27.0       3885.0           661.0
1       -118.30     34.26                43.0       1510.0           310.0
2       -117.81     33.78                27.0       3589.0           507.0
3       -118.36     33.82                28.0         67.0            15.0
4       -119.67     36.33                19.0       1241.0           244.0
...         ...       ...                 ...          ...             ...
2995    -119.86     34.42                23.0       1450.0           642.0
2996    -118.14     34.06                27.0       5257.0          1082.0
2997    -119.70     36.30                10.0        956.0           201.0
2998    -117.12     34.10                40.0         96.0            14.0
2999    -119.63     34.42                42.0       1765.0           263.0

      population  households  median_income  median_house_value
0         1537.0       606.0         6.6085            344700.0
1          809.0       277.0         3.5990            176500.0
2         1484.0       495.0         5.7934            270500.0
3           49.0        11.0         6.1359            330000.0
4          850.0       237.0         2.9375             81700.0
...          ...         ...            ...                 ...
2995      1258.0       607.0         1.1790            225000.0
2996      3496.0      1036.0         3.3906            237200.0
2997       693.0       220.0         2.2895             62000.0
2998        46.0        14.0         3.2708            162500.0
2999       753.0       260.0         8.5608            500001.0
```

```
[3000 rows x 9 columns]
```

Viewing Dataset

When datasets are large, we usually inspect the first few rows.

Useful functions:

head() → first rows
tail() → last rows

```
df.head()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	popu
0	-122.05	37.37	27.0	3885.0	661.0	
1	-118.30	34.26	43.0	1510.0	310.0	
2	-117.81	33.78	27.0	3589.0	507.0	
3	-118.36	33.82	28.0	67.0	15.0	
4	-119.67	36.33	19.0	1241.0	244.0	

```
df.tail()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	p
2995	-119.86	34.42	23.0	1450.0	642.0	
2996	-118.14	34.06	27.0	5257.0	1082.0	
2997	-119.70	36.30	10.0	956.0	201.0	
2998	-117.12	34.10	40.0	96.0	14.0	
2999	-119.63	34.42	42.0	1765.0	263.0	

Dataset Summary Information

Two very important functions:

info()

Shows:

- number of rows
- number of columns
- data types
- missing values

describe()

Provides statistical summary.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   longitude           3000 non-null   float64
 1   latitude            3000 non-null   float64
 2   housing_median_age  3000 non-null   float64
 3   total_rooms         3000 non-null   float64
 4   total_bedrooms      3000 non-null   float64
 5   population          3000 non-null   float64
 6   households          3000 non-null   float64
 7   median_income       3000 non-null   float64
 8   median_house_value  3000 non-null   float64
dtypes: float64(9)
memory usage: 211.1 KB
```

```
df.describe()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedroo
count	3000.000000	3000.00000	3000.000000	3000.000000	3000.0000
mean	-119.589200	35.63539	28.845333	2599.578667	529.9506
std	1.994936	2.12967	12.555396	2155.593332	415.6543
min	-124.180000	32.56000	1.000000	6.000000	2.0000
25%	-121.810000	33.93000	18.000000	1401.000000	291.0000
50%	-118.485000	34.27000	29.000000	2106.000000	437.0000
75%	-118.020000	37.69000	37.000000	3129.000000	636.0000
max	-114.490000	41.92000	52.000000	30450.000000	5419.0000

Exporting Data

Pandas also allows exporting datasets.

Export to CSV

df.to_csv("output.csv")

Export to Excel

df.to_excel("output.xlsx")

```
df.to_csv("exported_dataset.csv")
```

```
Start coding or generate with AI.
```